LINUX FOUNDATION COLLABORATIVE PROJECTS

Open vSwitch

Navigation

# VLANs

Q: What's a VLAN?

A: At the simplest level, a VLAN (short for "virtual LAN") is a way to partition a single switch into multiple switches. Suppose, for example, that you have two groups of machines, group A and group B. You want the machines in group A to be able to talk to each other, and you want the machine in group B to be able to talk to each other, but you don't want the machines in group A to be able to talk to the machines in group B. You can do this with two switches, by plugging the machines in group A into one switch and the machines in group B into the other switch.

If you only have one switch, then you can use VLANs to do the same thing, by configuring the ports for machines in group A as VLAN "access ports" for one VLAN and the ports for group B as "access ports" for a different VLAN. The switch will only forward packets between ports that are assigned to the same VLAN, so this effectively subdivides your single switch into two independent switches, one for each group of machines.

So far we haven't said anything about VLAN headers. With access ports, like we've described so far, no VLAN header is present in the Ethernet frame. This means that the machines (or switches) connected to access ports need not be aware that VLANs are involved, just like in the case where we use two different physical switches.

Now suppose that you have a whole bunch of switches in your network, instead of just one, and that some machines in group A are connected directly to both switches 1 and 2. To allow these machines to talk to each other, you could add an access port for group A's VLAN to switch 1 and another to switch 2, and then connect an Ethernet cable between those ports. That works fine, but it doesn't scale well as the number of switches and the number of VLANs increases, because you use up a lot of valuable switch ports just connecting together your VLANs.

This is where VLAN headers come in. Instead of using one cable and two ports per VLAN to connect a pair of switches, we configure a port on each switch as a VLAN "trunk port". Packets sent and received on a trunk port carry a VLAN header that says what VLAN the packet belongs to, so that only two ports total are required to connect the switches, regardless of the number of VLANs in use. Normally, only switches (either physical or virtual) are connected to a trunk port, not individual hosts, because individual hosts don't expect to see a VLAN header in the traffic that they receive.

None of the above discussion says anything about particular VLAN numbers. This is because VLAN numbers are completely arbitrary. One must only ensure that a given VLAN is numbered consistently throughout a network and that different VLANs are given different numbers. (That said, VLAN 0 is usually synonymous with a packet that has no VLAN header, and VLAN 4095 is reserved.)

Q: VLANs don't work.

A: Do you have VLANs enabled on the physical switch that OVS is attached to? Make sure that the port is configured to trunk the VLAN or VLANs that you are using with OVS.

Q: Outgoing VLAN-tagged traffic goes through OVS to my physical switch and to its destination host, but OVS seems to drop incoming return traffic.

A: It's possible that you have the VLAN configured on your physical switch as the "native" VLAN. In this mode, the switch treats incoming packets either tagged with the native VLAN or untagged as part of the native VLAN. It may also send outgoing packets in the native VLAN without a VLAN tag.

If this is the case, you have two choices:

- Change the physical switch port configuration to tag packets it forwards to OVS with the native VLAN instead of forwarding them untagged.
- Change the OVS configuration for the physical port to a native VLAN mode. For example, the following sets up a bridge with port eth0 in "native-tagged" mode in

Search 4.0 documentation

**Contents**

- Project
- Getting Started
- Tutorials
- How-to Guides
- Deep Dive
- Reference Guide
- Open vSwitch Internals
- Open vSwitch Documentation
- FAQ
  - Bareudp
  - Basic Configuration
  - Development
  - Implementation Details
  - General
  - Common Configuration Issues
  - Using OpenFlow
  - Quality of Service (QoS)
  - Releases
  - Terminology
  - VLANs
  - VXLANs

**Browse**

- General Index
- VXLANs
- Terminology

**Navigation**

- Open vSwitch 4.0.0 documentation
  - FAQ
    - VLANs

stable

VLAN 9:

```
$ ovs-vsctl add-br br0 $ ovs-vsctl add-port br0 eth0 tag=9
vlan_mode=native-tagged
```

In this situation, “native-untagged” mode will probably work equally well. Refer to the documentation for the Port table in ovs-vswitchd.conf.db(5) for more information.

Q: I added a pair of VMs on different VLANs, like this:

```
$ ovs-vsctl add-br br0
$ ovs-vsctl add-port br0 eth0
$ ovs-vsctl add-port br0 tap0 tag=9
$ ovs-vsctl add-port br0 tap1 tag=10
```

but the VMs can’t access each other, the external network, or the Internet.

A: It is to be expected that the VMs can’t access each other. VLANs are a means to partition a network. When you configured tap0 and tap1 as access ports for different VLANs, you indicated that they should be isolated from each other.

As for the external network and the Internet, it seems likely that the machines you are trying to access are not on VLAN 9 (or 10) and that the Internet is not available on VLAN 9 (or 10).

Q: I added a pair of VMs on the same VLAN, like this:

```
$ ovs-vsctl add-br br0
$ ovs-vsctl add-port br0 eth0
$ ovs-vsctl add-port br0 tap0 tag=9
$ ovs-vsctl add-port br0 tap1 tag=9
```

The VMs can access each other, but not the external network or the Internet.

A: It seems likely that the machines you are trying to access in the external network are not on VLAN 9 and that the Internet is not available on VLAN 9. Also, ensure VLAN 9 is set up as an allowed trunk VLAN on the upstream switch port to which eth0 is connected.

Q: Can I configure an IP address on a VLAN?

A: Yes. Use an “internal port” configured as an access port. For example, the following configures IP address 192.168.0.7 on VLAN 9. That is, OVS will forward packets from eth0 to 192.168.0.7 only if they have an 802.1Q header with VLAN 9. Conversely, traffic forwarded from 192.168.0.7 to eth0 will be tagged with an 802.1Q header with VLAN 9:

```
$ ovs-vsctl add-br br0
$ ovs-vsctl add-port br0 vlan9 tag=9 \
    -- set interface vlan9 type=internal
$ ip addr add 192.168.0.7/24 dev vlan9
$ ip link set vlan9 up
```

See also the following question.

Q: I configured one IP address on VLAN 0 and another on VLAN 9, like this:

```
$ ovs-vsctl add-br br0
$ ovs-vsctl add-port br0 eth0
$ ip addr add 192.168.0.5/24 dev br0
$ ip link set br0 up
$ ovs-vsctl add-port br0 vlan9 tag=9 -- set interface vlan9 type=internal
$ ip addr add 192.168.0.9/24 dev vlan9
$ ip link set vlan9 up
```

but other hosts that are only on VLAN 0 can reach the IP address configured on VLAN 9. What’s going on?

A: **RFC 1122 section 3.3.4.2 “Multihoming Requirements”** describes two approaches to IP address handling in Internet hosts:

- In the “Strong ES Model”, where an ES is a host (“End System”), an IP address is primarily associated with a particular interface. The host discards packets that

arrive on interface A if they are destined for an IP address that is configured on interface B. The host never sends packets from interface A using a source address configured on interface B.

- In the “Weak ES Model”, an IP address is primarily associated with a host. The host accepts packets that arrive on any interface if they are destined for any of the host’s IP addresses, even if the address is configured on some interface other than the one on which it arrived. The host does not restrict itself to sending packets from an IP address associated with the originating interface.

Linux uses the weak ES model. That means that when packets destined to the VLAN 9 IP address arrive on eth0 and are bridged to br0, the kernel IP stack accepts them there for the VLAN 9 IP address, even though they were not received on vlan9, the network device for vlan9.

To simulate the strong ES model on Linux, one may add iptables rule to filter packets based on source and destination address and adjust ARP configuration with sysctls.

BSD uses the strong ES model.

Q: My OpenFlow controller doesn’t see the VLANs that I expect.

A: The configuration for VLANs in the Open vSwitch database (e.g. via ovs-vsctl) only affects traffic that goes through Open vSwitch’s implementation of the OpenFlow “normal switching” action. By default, when Open vSwitch isn’t connected to a controller and nothing has been manually configured in the flow table, all traffic goes through the “normal switching” action. But, if you set up OpenFlow flows on your own, through a controller or using ovs-ofctl or through other means, then you have to implement VLAN handling yourself.

You can use “normal switching” as a component of your OpenFlow actions, e.g. by putting “normal” into the lists of actions on ovs-ofctl or by outputting to OFPP_NORMAL from an OpenFlow controller. In situations where this is not suitable, you can implement VLAN handling yourself, e.g.:

- If a packet comes in on an access port, and the flow table needs to send it out on a trunk port, then the flow can add the appropriate VLAN tag with the “mod_vlan_vid” action.
- If a packet comes in on a trunk port, and the flow table needs to send it out on an access port, then the flow can strip the VLAN tag with the “strip_vlan” action.

Q: I configured ports on a bridge as access ports with different VLAN tags, like this:

```
$ ovs-vsctl add-br br0
$ ovs-vsctl set-controller br0 tcp:192.168.0.10:6653
$ ovs-vsctl add-port br0 eth0
$ ovs-vsctl add-port br0 tap0 tag=9
$ ovs-vsctl add-port br0 tap1 tag=10
```

but the VMs running behind tap0 and tap1 can still communicate, that is, they are not isolated from each other even though they are on different VLANs.

A: Do you have a controller configured on br0 (as the commands above do)? If so, then this is a variant on the previous question, “My OpenFlow controller doesn’t see the VLANs that I expect,” and you can refer to the answer there for more information.

Q: How MAC learning works with VLANs?

A: Open vSwitch implements Independent VLAN Learning (IVL) for **`OFPP_NORMAL`** action, e.g. it logically has separate learning tables for each VLANs.

‹ Terminology VXLANs ›

© 2016-2023 A Linux Foundation Collaborative Project. All Rights Reserved.

Linux Foundation is a registered trademark of The Linux Foundation. Linux is a registered **trademark** of Linus Torvalds. Open vSwitch and OvS are trademarks of The Linux Foundation.

Please see our **privacy policy** and **terms of use**.